Ders 2 Çalışma Özeti

Ders 2 Çalışma Özeti

Genel Konular

- Dersin başında öğrenciler kameralarını açıp kendilerini tanıttı (yaklaşık ilk 20 dakika).
- Hocanın lisansüstü ders seçimi tavsiyesi.
- Eğitmenli öğrenmenin (supervised learning) genel yapısı: veri kümesi, giriş x, çıkış (etiket) y, fonksiyon f.
- Regresyon (regression) ve sınıflandırma (classification) ayrımı.
- Eğitmenli öğrenmenin iki aşaması: model eğitimi ve bilinmeyen örneğin tahmini.
- Veri tipleri ve özelliklerin ayırt ediciliği (tekrar).
- Karar sınırı (decision boundary): doğrusal (linear) ve doğrusal olmayan (non-linear).
- Karar ağaçları (decision trees): düğüm, dal, yaprak; ağaçtan kural çıkarma.
- Saflık / safsızlık (purity / impurity), entropi (entropy), bilgi kazancı (information gain).
- Tenis oynama (PlayTennis) örneğiyle adım adım ağaç kurma.
- ID3 algoritması, Occam'ın usturası (Occam's razor).
- Aşırı öğrenme / ezberleme (overfitting) ve nedenleri.
- Budama (pruning): azaltılmış hata budaması (reduced error pruning) ve kural sonrası budama (rule post-pruning). Örnekli anlatım bir sonraki derse kaldı.
- Karar ağaçlarının güçlü ve zayıf yanları.

Hocanın Özellikle Vurguladığı Kısımlar

- **Lisansüstü ders seçimi:** Birbirine çok benzeyen dersleri (makine öğrenmesi, veri madenciliği, yapay zeka) birlikte almayın; konuların yaklaşık %40'ı aynı. 7 dersin 3-4'ü uzmanlaşılacak alandan (ör. makine öğrenmesi + görüntü işleme ya da doğal dil işleme), en az 1-3'ü bilgisayar mühendisliğinin başka alanlarından (ör. güvenlik) olsun.
- **Veri kümesi her şeyin başı.** Eğitmenli öğrenme projesine (tez, Ar-Ge projesi) başlamadan önce iyi bir veri kümesi olduğundan ya da proje başlayana kadar toplanabileceğinden emin olun. Proje başladıktan sonra veri toplamaya güvenmeyin.
- **Başarı ölçülmeli.** "Model yüz tanıyor" demek yetmez; hangi başarıyla tanıdığı önemli. Yapay zeka problemleri deterministik değildir, %100 başarı genelde yoktur.
- **Önce problem türüne karar verin:** Sürekli çıkış gerekiyorsa regresyon, sınıf gerekiyorsa sınıflandırma.
- **Özellikler** sınıf içi örnekleri birbirine yaklaştırmalı, farklı sınıfların örneklerini uzaklaştırmalı. Anlamsız (useless) özellikler baştan elenmeli.
- **Tüm eğitim örneklerini doğru sınıflandırmaya çalışmak ezberlemeye götürür.** Genel bir çözüm hedeflenmeli.
- **Kök düğüme en ayırt edici (bilgi kazancı en yüksek) özellik** konur. Amaç hem yüksek başarı hem kısa (yüksekliği az) ağaç.
- **Sınıf dengesi:** Veri kümesinde sınıflardaki örnek sayıları mümkün olduğunca yakın olmalı (%50-%50, en kötü %40-%60 gibi). Aşırı dengesizlikte (ör. 1'e 40) az örnekli sınıfın karakteristiği öğrenilemeyebilir.
- **En iyi karar ağacını bulmak NP-zor (NP-hard) bir problemdir.** ID3 o anki en iyi seçimi yapan açgözlü (greedy) bir algoritmadır; bulduğu ağacın en iyisi olmama ihtimali vardır.
- Veri eğitim, doğrulama, test olarak üçe ayrılır. Doğrulama verisiyle model iyileştirilir (ör. budama), test verisiyle modele dokunmadan son başarı ölçülür. Performans ölçümü 1-2 hafta sonra ayrıca anlatılacak.

Kısa Tekrar Notları

- **Eğitmenli öğrenme veri kümesi:** $D = \{(x_i, y_i)\}_{i=1}^{N}$. x_i: i. örneğin özellikleri; y_i: etiketi (çıkışı).
- **Amaç:** $y \approx f(x)$ olacak, hatası en az olan bir f fonksiyonu bulmak.
- **Regresyon:** Giriş sürekli ya da ayrık, çıkış sürekli (ör. ev fiyatı, mısır hasılatı).
- **Sınıflandırma:** Giriş sürekli ya da ayrık, çıkış ayrık sınıf (ör. sahte / normal harcama, söylenen kelime, konuşan kişi).
- **Lojistik regresyon** adına rağmen buradaki regresyondan farklı değerlendirilir; ileride anlatılacak.
- **Karar ağacında:** düğüm = özellik, dal = özelliğin değeri, yaprak = sınıf.
- Ağaç, bir daldaki bütün örnekler aynı sınıfa ait olunca o dalda durur.
- **Entropi (2 sınıf):** $E(S) = -p_+ \log_2 p_+ - p_- \log_2 p_-$. Saf grupta 0, yarı yarıya dağılımda 1.
- **Bilgi kazancı:** $Gain(S, A) = E(S) - \sum_v \frac{|S_v|}{|S|} E(S_v)$.
- **ID3:** Iterative Dichotomiser 3. Yukarıdan aşağı (top-down), açgözlü (greedy) algoritma.
- **Overfitting:** Eğitim başarısı artarken test başarısının düşmesi. Karar ağacında ağaç büyüdükçe artar.
- **Önlem:** Ağacı az büyütmek (erken durdurma) ya da önce büyütüp sonra budamak.

Detaylı Açıklamalar

Tanışma ve Ders Seçimi Tavsiyesi

- Hoca, yüz yüze olsaydık yolda tanıyabilmek için öğrencilerden kendilerini tanıtmalarını istedi. Öğrenciler mezun oldukları bölümü, lisansüstü programlarını, çalıştıkları yeri ve dersi alma nedenlerini anlattı.
- Bir öğrencinin "umarım yapay zeka zehirlenmesine uğramam" sözü üzerine hoca ders seçimi tavsiyesi verdi:
  - Hoca yapay zekaya lisans 4. sınıfta kalp hastalıkları teşhisi için bir uzman sistem yazarak başlamış, yüksek lisans ve doktorasını görüntü işleme ağırlıklı yapmış.
  - Bilgisayar mühendisliğinin genel konularında ne kadar bilgi sahibi olursanız ileride o kadar faydasını görürsünüz.
  - Lisansüstü eğitim, lisansta görülen konularda uzmanlaşma ve derinleşme sağlar.
  - Birbirine çok benzeyen derslerden birini seçin, eksiklerinizi kendiniz tamamlayın.

Eğitmenli Öğrenmenin Yapısı

- Elimizde N örnekli bir veri kümesi (dataset) vardır.
- Her örnek iki bilgiden oluşur:
  - x: örneğin özellikleri (giriş).
  - y: örneğin çıkışı, yani etiketi (label), ait olduğu sınıf.
- Hedef: x girişleri için gerçek y çıkışlarına en yakın sonucu veren, yani hatası en az olan bir f fonksiyonu bulmak.
- Böylece bilinmeyen bir x örneği geldiğinde en yüksek başarıyla hangi sınıfa ait olduğu tahmin edilir.
- Hocanın örneği: Pandemi başında uçak yolcularına tavsiye sistemi kurmak isteyen bir proje, IATA ile çalışmasına rağmen elinde hazır veri kümesi olmadığı ve uluslararası uçuşlar kısıtlandığı için yürüyemedi.

Regresyon ve Sınıflandırma

Regresyon (regression)

- Giriş sürekli ya da ayrık olabilir, çıkış süreklidir. Sonuçta bir sınıf yoktur, bir değer tahmin edilir.
- Örnekler:
  - Bir bölgedeki 12 aylık yağış miktarına göre mısır hasılatını (ör. 400 kg, 500 kg) tahmin etmek. "Aralık'ta 1 ton yağış olursa kaç kilo mısır alırım?"

  - Evin metrekare, oda, salon, yaş bilgisine göre fiyatını tahmin etmek. “3+1, 140 m², 10 yaşında bir ev kaç lira?”
  - Sıcaklık tahmini.

Sınıflandırma (classification)

- Giriş sürekli ya da ayrık olabilir, çıkış ayrık bir sınıftır.
- Örnekler:
  - Evin “ucuz / pahalı” olduğunu söylemek.
  - Kredi kartı harcamasının (tutar, yer, kart tipi) sahte mi normal mi olduğu: 2 sınıf.
  - Ses sinyalinden söylenen kelimeyi bulmak (“kırmızı”, “yeşil”, “defter”).
  - Sesten konuşan kişiyi tanımak. Her kişi bir sınıftır.
- Ev fiyatı probleminde “ucuz / pahalı” yetmez, fiyat gerekir. Bu yüzden regresyon daha uygundur. Problemin ihtiyacına göre karar verilir.

Eğitmenli Öğrenmenin İki Aşaması

1. **Model eğitimi:** D örnekleriyle (girişler x, çıkışlar y) bir model eğitilir. Model örnekleri: k-en yakın komşu (k-NN), karar ağacı, destek vektör makineleri (SVM), derin öğrenme.
2. **Tahmin:** Sınıfı (ya da regresyonda değeri) bilinmeyen yeni bir x örneği modelden geçirilir, y tahmin edilir.

Veri Tipleri ve Özellik Seçimi (Tekrar)

- **Ordinal:** Arasında ilişki kurulabilen, ölçülebilen, sıralanabilen sayılar (yaş, boy, sıcaklık). 30 kg ile 33 kg arasındaki fark anlamlıdır.
- **Kategorik (nominal):** Farklı değerler alır ama birbirine göre ölçülebilir üstünlüğü ya da mesafesi yoktur (kartın kullanıldığı şehir: İstanbul, Ankara, İzmir).
- **Anlamsız (useless):** Kimlik numarası, ad soyad gibi kişiye özel bilgiler. Öğrenmede kullanılmamalı, baştan elenmeli.
- Özellik seçimi sınıflandırma başarısını doğrudan etkiler. Ayrı bir derste anlatılacak.
- Örnek: Türkiye’de yüz tanımada göz rengi ayırt edici değildir, çünkü çoğu kişinin gözü kahverengidir.

Karar Sınırları

- Eğitmenli öğrenmenin amacı sınıfları ayıran en iyi karar sınırını oluşturmaktır. Bütün modeller, yapılarına bağlı olarak bir şekilde karar sınırı oluşturur.
- Bilinmeyen bir örnek, sınırın hangi tarafında kalıyorsa o sınıfa atanır.
- Slayttaki örnekler:
  - İki sınav notuna göre kabul edilen / edilmeyen öğrenciler: doğrusal (linear) karar sınırı.
  - İçeride yeşil, dışarıda kırmızı örnekler: doğrusal olmayan (non-linear) karar sınırı.
- İki örnekte de %100 başarı yoktur. Bütün örnekleri doğru ayırmaya çalışmak ezberlemeye gider.
- Doğrusal olmayan sınırı matematiksel olarak modellemek zordur. Karar ağacı bunu eğer-ise (if) kontrolleriyle kolaylaştırır.

Karar Ağaçları

Mantık

- Karar ağacı özellik uzayını eksenlere dik sınırlarla bölgelere ayırır. Her bölgede tek bir sınıfın örnekleri kalır.
- Slayttaki iki özellikli (x_1, x_2), iki sınıflı (0 ve 1) örnek:

```
x2 < 3 ?
├─ evet: x1 < 4 ?
│        ├─ evet  -> sınıf 0
│        └─ hayır -> sınıf 1
└─ hayır: x1 < 3 ?
          ├─ evet: x2 < 4 ?
```

```
│        ├─ evet -> sınıf 0
│        └─ ...
└─ ...
```

- Not: Ağacın kalan dalları altyazıda okunmuyor, yalnız slaytta var.

Yapı

- **Düğüm (node):** Bir özellik (ör. Outlook, Humidity).
- **Dal (branch):** Özelliğin aldığı bir değer (ör. Outlook = Sunny, Overcast, Rain).
- **Yaprak (leaf):** Sınıf. Bir daldaki bütün örnekler aynı sınıfa ait olunca ağaç orada biter.
- Karar ağacının birçok modelden farklı ve avantajlı yanı: **nominal (kategorik) özellikler doğrudan kullanılabilir** (ör. renk = kırmızı / yeşil / mavi).
- Sürekli özellikler eşik değeriyle bölünür (ör. sıcaklık > 20 °C ise true, değilse false). Eşiğin nasıl seçileceği ayrıca anlatılacak.

Ağaçtan kural çıkarma

- Kurulan ağaç bilgisayarda kural grubu olarak kullanılır. PlayTennis ağacı için:

```
EĞER (Outlook = Sunny VE Humidity = Normal)
   VEYA (Outlook = Overcast)
   VEYA (Outlook = Rain VE Wind = Weak)          İSE PlayTennis = Yes

EĞER (Outlook = Sunny VE Humidity = High)
   VEYA (Outlook = Rain VE Wind = Strong)        İSE PlayTennis = No
```

PlayTennis Örneği (Tom Mitchell)

- Hocanın anlattığı hikaye: Utangaç bir genç, bir kızı tenise davet etmek istiyor. 14 gün boyunca hava koşullarını ve kızın tenis oynayıp oynamadığını not alıyor. 15. günün koşullarına göre davet edip etmeyeceğine karar verecek.
- Örnek, Tom Mitchell'ın "Machine Learning" kitabından. Karar ağaçları için çok kullanılır.
- 4 özellik, 2 sınıf:

Özellik	Değerler
Outlook (hava durumu)	Sunny, Overcast, Rain
Temperature (sıcaklık)	Hot, Mild, Cool
Humidity (nem)	High, Normal
Wind (rüzgar)	Weak, Strong
Sınıf: PlayTennis	Yes, No

- 14 günlük veri (Mitchell'ın kitabındaki standart tablo; hoca slayttan gösteriyor):

Gün	Outlook	Temperature	Humidity	Wind	PlayTennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

En İyi Özellik: Saflık ve Bilgi Kazancı

- Bir ağacı rastgele özellik seçerek de kurmak mümkün ama bu etkili değil.
- **En iyi özellik**, örnekleri olabildiğince saf (pure) gruplara bölen özelliktir.
- Slayttaki örnek (2 sınıf):
  - "Ağırlık > 50K" özelliği: iki tarafta da çoğunluk bir sınıfta ama karışık. Bir kontrol daha gerekir.
  - "Çalışıyor mu (employed)" özelliği: true tarafında yalnız bir sınıf, false tarafında yalnız diğer sınıf. Ağaç hemen biter.
  - Daha iyi kök özelliği "employed"dır, çünkü daha küçük ve daha az karmaşık bir ağaç verir.
- **Safsızlık (impurity):** Bir grubun farklı sınıflardan örneklerle ne kadar karışık olduğu. Tamamen saf grupta tüm örnekler aynı sınıftadır; en karışık grupta iki sınıf yarı yarıyadır.
- **Bilgi kazancı (information gain):** Bir özelliğin örnekleri sınıflara ayırmada ne kadar önemli olduğunu ölçer.

Entropi

- Entropi, bir kümedeki örnekleri kodlamak için gereken en az bit sayısıdır.
- c sınıf için, p_i bir örneğin i. sınıfa ait olma olasılığı olmak üzere:

$$E(S) = -\sum_{i=1}^{c} p_i \log_2 p_i$$

- İki sınıf (artı / eksi) için:

$$E(S) = -p_+ \log_2 p_+ - p_- \log_2 p_-$$

- İki sınıfta entropi 0 ile 1 arasında değişir:
  - Bütün örnekler aynı sınıftaysa ($p_+ = 0$ ya da $p_+ = 1$): $E = 0$.
  - Örnekler yarı yarıya dağılmışsa ($p_+ = 0.5$): $E = 1$.
- Örnekler sınıflara ne kadar eşit dağılmışsa entropi o kadar yüksektir.

Örnek 1: Toplam entropi

- 64 örnek: 29 artı, 35 eksi.

$$E(S) = -\frac{29}{64} \log_2 \frac{29}{64} - \frac{35}{64} \log_2 \frac{35}{64} \approx 0.99$$

- İki sınıfta yaklaşık eşit örnek olduğu için entropi 1'e yakın.

Bilgi Kazancı

$$Gain(S, A) = E(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} E(S_v)$$

- S_v: A özelliğinin değeri v olan örnekler.
- Toplam entropiden, özelliğin her değeri için ağırlıklı (örnek oranıyla çarpılmış) entropiler çıkarılır.
- Toplam entropi $E(S)$, tüm örnekler için hep aynıdır; bir öğrencinin sorusu üzerine hoca bunu ayrıca açıkladı.
- Her özellik için kazanç hesaplanır. Kazancı en yüksek özellik köke konur.

Örnek 2: A1 mi A2 mi köke konmalı?

- Toplam: 64 örnek, [29+, 35−], $E(S) \approx 0.99$.

Özellik	Değer	Örnekler	Entropi
A1	true	[21+, 5−] (26 örnek)	0.71
A1	false	[8+, 30−] (38 örnek)	0.74
A2	true	[18+, 33−] (51 örnek)	≈ 0.94
A2	false	[11+, 2−] (13 örnek)	≈ 0.62

$$E([21+, 5-]) = -\frac{21}{26}\log_2\frac{21}{26} - \frac{5}{26}\log_2\frac{5}{26} \approx 0.71$$

$$Gain(S, A1) = 0.99 - \frac{26}{64}(0.71) - \frac{38}{64}(0.74) \approx 0.27$$

$$Gain(S, A2) \approx 0.12$$

- Sonuç: **A1 köke konur.**
- Sezgisel yorum: A1'in iki dalında da bir sınıf belirgin çoğunlukta (21'e 5, 8'e 30). A2'nin true dalı neredeyse yarı yarıya (18'e 33), daha fazla bölme gerektirebilir.
- Bir öğrencinin sorusu: En ideal durum, A1 true dalındaki 26 örneğin hepsinin bir sınıfta, false dalındaki 38 örneğin hepsinin öbür sınıfta olmasıdır. O zaman problem tek adımda çözülürdü. Gerçekte dallar karışıktır ve ağaç aşağı doğru devam eder.

Örnek 3: PlayTennis için kök seçimi

- Toplam: 14 örnek, [9 Yes, 5 No].

$$E(S) = -\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14} = 0.940$$

- **Wind:**
  - Weak: 8 gün [6 Yes, 2 No], $E = 0.811$.
  - Strong: 6 gün [3 Yes, 3 No], $E = 1.0$.

$$Gain(S, Wind) = 0.940 - \frac{8}{14}(0.811) - \frac{6}{14}(1.0) = 0.048$$

- **Humidity:**
  - High: 7 gün [3 Yes, 4 No], $E \approx 0.985$.
  - Normal: 7 gün [6 Yes, 1 No], $E \approx 0.592$.

$$Gain(S, Humidity) = 0.940 - \frac{7}{14}(0.985) - \frac{7}{14}(0.592) = 0.151$$

- **Outlook:**
  - Sunny: 5 gün [2 Yes, 3 No].
  - Overcast: 4 gün [4 Yes, 0 No].
  - Rain: 5 gün [3 Yes, 2 No].
  - $Gain(S, Outlook) \approx 0.246$ (hoca 0.247 diyor).
- **Temperature:** $Gain(S, Temperature) \approx 0.029$ (altyazıda sözlü olarak verilmiyor; Mitchell'ın kitabındaki değer).
- En yüksek kazanç Outlook'ta. **Outlook köke konur.**

Alt ağaçlar (rekürsif)

- **Sunny dalı** (5 örnek): Kalan özelliklerin kazançları hesaplanır.
  - $Gain(Humidity) = 0.970$, $Gain(Temperature) = 0.570$, $Gain(Wind) = 0.019$.
  - Humidity seçilir. High ise hepsi No, Normal ise hepsi Yes. Dal biter.
- **Overcast dalı** (4 örnek): Hepsi Yes. Dal hemen biter.

- **Rain dalı** (5 örnek, [3 Yes, 2 No]): Aynı hesap tekrarlanır. Sonuçta Wind seçilir: Weak ise Yes, Strong ise No.
- Son ağaç:

```
Outlook
├─ Sunny    -> Humidity
│              ├─ High   -> No
│              └─ Normal -> Yes
├─ Overcast -> Yes
└─ Rain     -> Wind
               ├─ Weak   -> Yes
               └─ Strong -> No
```

- Bir öğrencinin sorusu: Sunny ve Rain dallarında aynı özellik (ör. Humidity) en yüksek kazancı verebilir mi? Evet. Dallar birbirinden bağımsız değerlendirilir. Bir dalda yukarıdan gelirken o yol üzerinde seçilmemiş özellikler kullanılır. Bir yol üzerinde bir özellik ikinci kez kullanılmaz.

ID3 Algoritması

- Ross Quinlan tarafından geliştirildi.
- Ad: **I**terative **D**ichotomiser **3**.
  - Iterative: Aynı işlem her alt ağaç için tekrar edilir.
  - Dichotomiser: Bölen; veri her adımda alt parçalara bölünür.
- **Yukarıdan aşağı (top-down):** Ağaç kökten başlayarak kurulur.
- **Açgözlü (greedy):** Her adımda o an için en iyi özellik seçilir. Bu yüzden bulunan ağacın en iyisi olma garantisi yoktur. Belki A2 seçilseydi bir sonraki adımda ağaç hemen bitecekti.

```
ID3(örnekler, özellikler):
  1. Tüm örnekler aynı sınıftaysa: o sınıfla etiketli yaprak döndür.
  2. A <- bilgi kazancı en yüksek özellik; A'yı düğüm yap.
  3. A'nın her v değeri için:
       S_v <- A = v olan örnekler
       alt ağaç <- ID3(S_v, özellikler - {A})
  4. Düğümü döndür.
```

- Kurulan ağaç kurallara çevrilir. Bilinmeyen örnek kurallara göre sınıflandırılır. Örnek: Outlook = Sunny ve Humidity = Normal ise tenis oynanacak; Outlook = Overcast ise tenis oynanacak.

Occam'ın Usturası (Occam's Razor)

- ID3 bu felsefeye dayanır.
- Latince sözün İngilizcesi: "Entities are not to be multiplied beyond necessity."
- Anlamı: Birden fazla çözüm aynı sonuca ulaşıyorsa en basit, en az karmaşık olanı seçin. "En iyi çözüm en basit çözümdür."
- Karar ağacında: Aynı başarıyı veren ağaçlardan en kısa olanı tercih edilir.

Aşırı Öğrenme (Overfitting)

- Overfitting, eğitim örneklerini gereğinden fazla öğrenmektir. Türkçede "ezberleme" denir.
- **Örnek:** Kız ve erkek öğrencileri boy ve kiloya göre ayırırken 1,90 m boyunda 80 kg bir kız öğrenci erkekler tarafında kalır.
  - Model genel olarak iyi bir karar sınırı çizerse bu örneği yanlış sınıflandırır ama genelleme iyi olur.
  - Model "bütün eğitim örnekleri doğru sınıflandırılmalı" diye hırsla bu örneğin etrafından da sınır geçirirse, bu örneğin yakınına gelen yeni bir örnek (çevresi hep diğer sınıf olmasına rağmen) yanlış sınıfa atanır.
  - Eğitimde %100 başarı, testte düşük başarı.

- **Nedenleri:**
  - Gürültülü (parazitli, noisy) örnekler: Bütün özellik değerleri aynı olan iki örneğin farklı sınıflarda olması (ör. aynı hava koşullarında bir gün oynamış, bir gün oynamamış).
  - Aykırı değerler (outlier): Bir örneğin diğer sınıfın örneklerinin arasına düşmesi.
  - Hatalı etiketleme: Etiketleyen kişinin yanlış (ya da tartışmalı) etiket vermesi. Çok sık görülür.
- **Karar ağacında etkisi:** PlayTennis verisine "Outlook = Sunny, Humidity = Normal iken oynamadı" diye gürültülü bir örnek eklenirse, Humidity = Normal dalı artık saf olmaz ve ağaç aşağı doğru uzamaya devam eder.
- **Slayttaki grafik (düğüm sayısı - doğruluk):**
  - Eğitim verisinde düğüm sayısı arttıkça doğruluk sürekli artar.
  - Test verisinde doğruluk bir noktaya kadar artar, sonra düşmeye başlar. Bu nokta overfitting'in başladığı yerdir.
- **Önlem için iki yaklaşım:**
  1. Ağacı kurarken fazla büyütmemek (erken durdurma).
  2. Önce ağacı tam kurmak, sonra budamak (pruning).
- Ağacın iyi olup olmadığı doğrulama verisiyle ölçülen başarıya bakılarak anlaşılır.

Budama (Pruning)

Azaltılmış hata budaması (reduced error pruning)

1. Veri eğitim ve doğrulama olarak ikiye ayrılır.
2. Ağaç eğitim verisiyle tam olarak kurulur.
3. Ağaç yapraklardan köke doğru budanır. Bir düğüm budanınca alt ağacı silinir; düğüm, o noktadaki örneklerin çoğunluk sınıfıyla etiketli bir yaprak olur.
4. Doğrulama başarısı düşmediği sürece budamaya devam edilir. Budama başarıyı düşürmeye başlayınca durulur.

- Hocanın örneği (altyazıda kısmen anlaşılıyor): Bir dalda eğitim verisinde "Color = Red ise pozitif, Color = Blue ise negatif" diye bir bölme yapılmış. Doğrulama verisinde Red iken [1+, 3−], Blue iken [1+, 1−] çıkıyor; toplam 4 yanlış sınıflandırma var. Bu noktadaki örneklerin çoğunluğu negatif. Color düğümü kaldırılıp dal doğrudan "negatif" yaprağına çevrilirse doğrulama başarısı artar. Bu yüzden o alt ağaç budanır.
- Slayttaki grafikte ağaç geriye doğru yaklaşık 20 düğüm budandığında test doğruluğu en yüksek değere (yaklaşık 0.73) çıkıyor.

Kural sonrası budama (rule post-pruning)

- Önce ağaç kurallara çevrilir. Sonra her kuralın koşulları birbirinden bağımsız olarak çıkarılıp başarı ölçülür.
- Hoca bu yöntemin zor olduğunu ve az kullanıldığını söylüyor. Slaytta "en sık kullanılan yöntem" yazsa da hocaya göre azaltılmış hata budaması daha çok kullanılıyor.
- Budama örnekleri bir sonraki derste anlatılacak.

Karar Ağaçlarının Güçlü ve Zayıf Yanları

Güçlü yanlar

- Hem basit hem başarılı bir model.
- Hızlı: Ağacın bir dalına girildiğinde diğer dallara hiç bakılmaz; büyük bir bilgi kullanılmadan elenmiş olur. Özellikle bilinmeyen örneğin sınıflandırılması (test) aşamasında hızlıdır.
- Ağaç kurallara çevrilebilir; anlaşılır.
- Nominal özellikleri doğrudan kullanabilir.
- Gürültülü veri problemi budama ile çözülebilir.

Zayıf yanlar

- En iyi karar ağacını bulma garantisi yok; eldeki imkanlarla en iyisi bulunur.
- Özellik sayısı 20-30'a çıkınca ağaç çok karmaşıklaşır; anlaşılması, kurala çevrilmesi, takibi zorlaşır.
- Özellik vektörünün (özellik sayısının) baştan biliniyor olması gerekir.

- Artımlı (incremental) değildir: Yeni örnekler ya da yeni özellikler gelince mevcut ağaca eklenemez, ağacın baştan kurulması gerekir.

Ders Sonu

- Hoca slaytları kendine ait bir sitesi olmadığı için AVESİS'teki sayfasından paylaştığını söylüyor. Ders kayıtlarına ders sonrası erişilebilir.
- Uzaktan eğitim biriminden bir yetkili, LMS'in dosya sunucusunun ayrıldığını ve materyal yükleme alanının bir hafta içinde açılacağını söylüyor.
- Bir sonraki ders budama örneklerle anlatılarak devam edilecek.

Ders sonrası notlar

- Altyazıda "Rose Queen" olarak geçen ad, ID3'ü geliştiren **Ross Quinlan**'dır (1986).
- Hoca en iyi karar ağacını bulmayı "NP problem, yani polinom zamanda çözülemeyen problem" diye tanımlıyor. Doğrusu: En küçük / en iyi karar ağacını bulmak **NP-tam (NP-complete) / NP-zor** bir problemdir; bilinen polinom zamanlı bir algoritması yoktur. "NP" kendi başına "polinom zamanda çözülemez" anlamına gelmez.
- Hoca bir yerde entropi için "örnekler ne kadar homojen sınıflara dağılmışsa o kadar yüksek" diyor. Kastedilen: örnekler sınıflara ne kadar **eşit** dağılmışsa entropi o kadar yüksektir. Bir sınıfta toplanmış (saf) grupta entropi 0'dır.
- İki sınıflı problemde entropi 0-1 arasındadır. c sınıfta en büyük değer $\log_2 c$'dir.
- Hoca entropiyi tanımlarken önce "N tane örnek / özellik" dedi, sonra kendisi "N tane sınıf" diye düzeltti. Doğrusu: c (ya da N) tane **sınıf**.
- Hoca PlayTennis'te Outlook dağılımını önce "5 Sunny, 4 Overcast, 4 Rain" dedi, sonra "5 Rain" diye düzeltti. Doğrusu: 5 Sunny, 4 Overcast, 5 Rain.
- Occam'ın usturasının sözü hocanın çevirisinde "bilgilere gereğinden fazla önem vermeyin" diye geçiyor. Sözün tam çevirisi: "Varlıklar gereğinden fazla çoğaltılmamalıdır."
- Altyazıda "Ayata" IATA olmalıdır.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.